

1) Abstract

In disaster response scenarios, mobile robots reduce risks for human operators while enabling exploration of hazardous environments. Their effectiveness, however, strongly depends on the reliability and latency of the communication infrastructure linking the operator to the robot. This thesis investigates the feasibility of teleoperating a *ROSbot XL* within a *ROS 2* environment under varying network constraints. A *Gazebo*-based simulation integrated with *ROS 2 Jazzy* is used, where the robot performs a standardized start/stop task in response to obstacles, with latency and stopping distance serving as key performance indicators. Network degradation is emulated through artificial latency and packet loss injection via *netns*, reflecting conditions analogous to 5G, Wi-Fi, and satellite connectivity. *MATLAB* is employed for data analysis of reaction times and control stability. The results highlight the trade-offs between low-latency terrestrial networks, which enable precise and reliable teleoperation, and high-latency satellite links, which compromise responsiveness and safety. This work provides a reproducible methodology for assessing the resilience of robotic teleoperation systems in challenging communication environments.

2) Intro

In recent years, both natural and man-made disasters have underscored the critical need for rapid and reliable emergency response. Traditional rescue operations often pose significant risks to both rescuers and victims, and in many situations may be infeasible. In this context, rescue robots have emerged as invaluable tools for ensuring the success of rescue missions. Teleoperated robots, in particular, provide a direct interface between the operator and the machine, enabling precise control over movement and task execution while leveraging onboard sensors to navigate complex and unpredictable environments.

The effectiveness of such interventions is inherently linked to the reliability of the communication infrastructure connecting operators to their machines. Disaster-stricken areas often experience compromised terrestrial networks due to physical damage, congestion, or lack of coverage, necessitating resilient connectivity solutions. Local wireless networks, such as Wi-Fi or ad-hoc 5G cells, offer low-latency connections suitable for fine-grained control but are limited in coverage. Satellite networks provide global reach, allowing remote operators to control robots from afar, yet they introduce higher latency and increased packet loss. These constraints directly impact the safety and responsiveness of teleoperated robots, particularly for critical control actions.

Within this framework the start/stop control is a fundamental and safety-critical one: Ensuring that a robot can reliably move in a delicate environment where even the slightest error might cause irreparable damage is a fundamental task to reliably achieve under network impaired conditions. While seemingly straightforward, start/stop control is highly sensitive to network-induced delays, jitter, and packet loss. Understanding the quantitative effects of these impairments is thus crucial for both practical deployment and the design of resilient control strategies.

This thesis focuses on the ROSbot XL, a research-oriented mobile platform, to investigate the feasibility and robustness of start/stop teleoperation under realistic network constraints. The study employs a controlled simulation environment based on ROS 2 and Gazebo to reproduce reproducible and systematically varied network conditions. Specific impairments, including latency, packet loss, and transmission timeouts, are introduced to emulate the challenges encountered in disaster-response and remote operations. Complementary MATLAB-based analysis enables the evaluation of performance metrics such as reaction time, braking distance, minimum safe distance, and crash probability across multiple trials.

the operator must ensure that the robot can reliably receive and act upon simple “move” and “halt” commands even under adverse network conditions. An unreliable start/stop link increases the risk of crashes or mission failure, especially when the robot is navigating near unstable or fragile obstacles. This makes the study of communication reliability in such minimal control schemes both practically relevant and scientifically interesting.

Starting from a probabilistic description of the problem, this work aims to construct a rigorous mathematical framework that captures the interplay between network reliability and robot safety in teleoperation. Specifically, the study focuses on (i) quantifying crash probabilities under varying network conditions, (ii) determining bounds on allowable network impairments, and (iii) validating these results through both controlled simulation environments and experimental tests with the ROSbot XL platform. Additionally, (iv) the research explores passive strategies to mitigate operational risks in real time whenever these bounds are exceeded, thereby ensuring robust and safe teleoperation even under suboptimal communication conditions.

3) State of the Art

4) Background And Tools

This chapter introduces the foundational tools and concepts underlying the experiments conducted in this work. Specifically, it describes *ROS 2*, *Gazebo*, the *ROSbot XL*, and *Linux namespaces (netns)*, highlighting how these components are combined to study the effects of network impairments on robot teleoperation. The chapter establishes the technical context for evaluating start/stop control under controlled but realistic conditions.

ROS 2 and Gazebo

The Robot Operating System (*ROS*) 2 is a middleware framework that standardizes communication between a robot's components. Each functional unit of the robot is implemented as a node, and nodes communicate asynchronously using a publish/subscribe mechanism. This design simplifies integration, improves scalability, and allows precise control over timing, which is critical when analyzing delays and packet losses in teleoperation. *ROS* 2 also provides Quality-of-Service (QoS) policies to manage reliability, latency, and message delivery guarantees. In this work, these QoS features are leveraged to emulate and measure the impact of adverse network conditions on the responsiveness of start/stop commands.

Gazebo is a high-fidelity simulation environment that models robot dynamics, sensors, and physical interactions with the environment. Its use ensures reproducibility across experiments, allowing identical conditions to be maintained while varying network parameters systematically. By integrating *ROS 2* with *Gazebo*, the *ROSbot XL* can be simulated in realistic scenarios where the only variables are network-induced impairments such as latency, packet loss, and transmission timeouts. This controlled approach provides a clean experimental framework for isolating the effects of communication reliability on robot safety and performance.

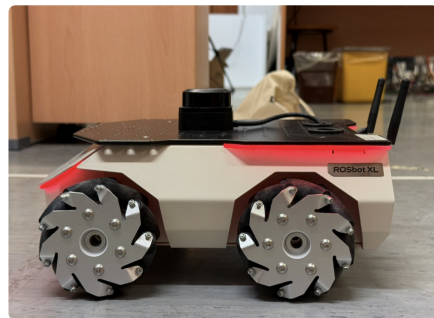
TODO ADD IMAGE OF ROBOT IN GAZEBO

ROSBot XL

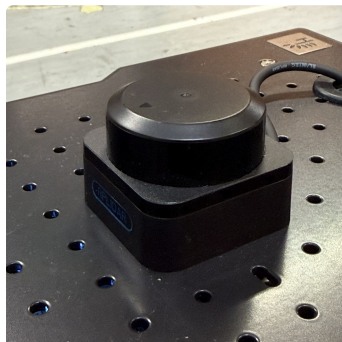
The *ROSbot XL* is a research-oriented mobile robot platform equipped with a 4x4 drive, LiDAR, IMU, and onboard computing capabilities. Its hardware design and sensor suite make it well-suited for start/stop teleoperation experiments, bridging the gap between simulation and real-world deployment. Experiments on the *ROSbot XL* provide empirical validation of the simulated results, ensuring that insights gained in *Gazebo* translate to the behavior of a physical robot operating under realistic conditions.



ROSBot XL



ROSBot XL Side



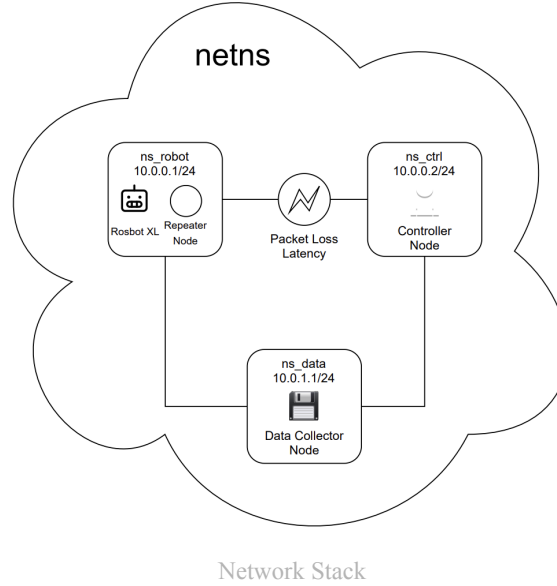
Linux Namespaces (netns)

Linux namespaces (netns) are a kernel feature that provides logically isolated network stacks. They enable multiple virtual networks to coexist on a single machine, allowing complex network topologies to be simulated. In this work, namespaces are used to emulate unstable links, packet loss, and variable latencies, providing a controlled environment for testing the robot's behavior under different network conditions.

The combination of *ROS 2*, *Gazebo*, the ROSbot XL, and *netns* allows the experiments to directly address the central aim of this thesis: quantifying the relationship between network reliability and robot safety under start/stop teleoperation. The simulation environment provides a safe, reproducible setting to systematically vary network impairments, while the ROSbot XL validates that these findings hold in practice. This integrated setup enables rigorous measurement of performance metrics such as reaction time, braking distance, minimum safe distance, and crash probability, which together inform the development of robust control strategies and passive mitigation mechanisms.

5) Test Environment and System Design

Gazebo provides a stable and physically accurate simulation of the *ROSbot XL*'s operation. As with any rigorous testing setup, reproducibility and accurate data collection are essential for the correct evaluation of results. Since both the robot and controller data are critical, a dedicated data collector with direct access to both namespaces is employed, while the unstable link between the robot and controller remains in place. The following diagram illustrates the *netns* virtual network stack environment used for this experiment.



To ensure reproducibility and consistency across trials, each experiment was conducted in a freshly instantiated *Gazebo* environment. The robot was initialized at position $(0, 0, 0)$ with the obstacle placed $6.2, [m]$ in front of it. This configuration guarantees that the robot reaches its maximum velocity before interaction with the obstacle. All nodes were launched simultaneously at the start of each test to standardize initial conditions.

In order to simulate a fully remote teleoperation scenario, no direct communication between the robot's *LiDAR* sensor and its motor drivers was permitted. All motion control was instead routed through the networked communication layer. Since *ROS 2* does not natively define explicit start and stop primitives, these signals were formalized as follows:

- A **start signal** is equivalent to a go command and is represented by publishing a `/cmd_vel` topic with a speed along the x-axis corresponding to the desired robot speed.
- A **brake signal** is represented by the same topic with zero speed in all directions.

Since commands must be continuously published for the robot to follow them correctly, a **repeater node** on the robot is implemented to:

- Keep publishing brake signals once received, stopping them only when a start signal is intercepted.
- Implement a **failsafe timeout (TO)**, which automatically triggers a brake if no command is received within a set interval, ensuring safe operation during connection failures.

The controller transmits commands at a frequency of $f = 10 [Hz]$ (interval $T = 100[ms]$), chosen as a compromise between communication efficiency and control responsiveness. Network conditions were emulated using symmetrical *netns* interfaces, such that both directions experienced identical impairments. If p denotes the packet loss probability and τ the one-way latency of each link, the effective end-to-end characteristics are

$$p_{tot} = 1 - (1 - p)^2, \quad \tau_{tot} = 2\tau.$$

Before starting, the controller pings the robot multiple times to estimate one-way packet loss and latency. During operation, it simultaneously analyzes the *LiDAR* data to estimate the robot's distance to the obstacle.

Two brake logic implementations are tested:

1. **Standard implementation:** The controller sends brake signals once the robot crosses a fixed brake distance threshold.
2. **Safe Margin Automatic Response Tool (SMART) implementation:** Before movement, an optimized brake distance threshold is calculated based on network conditions to guarantee a predefined crash probability, allowing the system to adapt ensuring safe operation under varying packet loss and latency.

The experimental campaign proceeds in two stages. First, the standard implementation is tested under ideal network conditions to establish baseline performance metrics. These results serve as ground truth for the construction and validation of the probabilistic model underpinning the SMART implementation, which is then evaluated under variable network impairments.

6) Mathematical Model

To enable the development of the smart bot implementation, it is first necessary to establish a mathematical model that captures the essential dynamics of the system. A rigorous model is not only a design tool but also a means to understand how the robot's control logic interacts with both its physical environment and the underlying communication network.

This model serves two main purposes. First, it provides the foundation for computing adaptive braking thresholds that allow the robot to operate intelligently under varying network conditions. By quantifying the relationship between obstacle distance, braking dynamics, and communication delay, the model enables the system to anticipate potential risks and make informed decisions rather than relying on fixed safety margins. Second, it offers a structured reference framework against which the results obtained through simulation and real-world experiments can be compared and validated. In this sense, the mathematical model acts as a bridge between theoretical expectations and practical observations, highlighting any discrepancies and guiding improvements in both design and testing methodology.

This chapter relies on a set of parameters assumed to be known by the controller:

- Robot parameters: the **sensor offset from the front** L [cm] and the **braking distance** d_{brake} [cm].
- Design parameters: the **command send period** T [ms], the **brake threshold** S [cm], **maximum speed** v_{max} [cm/s], and the **safety margin** m [cm] required at the end of braking.
- Network parameters: the **one-way network delay** τ , and the **one-way packet loss probability** p .

Theorem (Expected Time Crash Bound).

Safe operation is guaranteed if the expected communication delay does not exceed half of the available reaction time:

$$2\mathbb{E}[t] \leq \Delta t$$

Where

$$\mathbb{E}[t] = \tau + T \cdot \frac{p + 1}{2(1 - p)}$$

$$\Delta t = 1000 \cdot \frac{S - L - m - d_{brake}}{v_{max}}$$

The *Expected Time Crash Bound* theorem establishes a fundamental criterion for safe teleoperation under network-impaired conditions. It links the expected communication delay between the operator and the robot to the physical capabilities and configuration of the robot itself. This bound is crucial as it allows the integration of network reliability through parameters such as latency and packet loss probability directly into the safety analysis, rather than treating communication impairments as an abstract or qualitative concern. By expressing safety in terms of expected delay rather than worst-case scenarios, it enables more practical and adaptable control strategies that balance responsiveness with robustness, rather than overly conservative margins that may unnecessarily limit operational efficiency.

Theorem (Crash Probability).

The probability of a crash occurring due to network delay and packet loss can be expressed as:

$$P_{crash}(p, \tau) = p^{N_\tau} (1 + (1 - p)N_\tau)$$

Where

$$N_{\tau} = \left\lfloor \frac{\Delta t - 2\tau}{T} \right\rfloor$$

The Crash Probability theorem represents the main parameter of the mathematical framework for teleoperated robotic safety. It provides an explicit, quantitative relationship between network impairments.

This result is fundamental for several reasons. First, it translates abstract network metrics into a concrete probability of system failure, enabling a rigorous assessment of operational risk. Second, it allows the derivation of safe operating thresholds: given a target crash probability, one can directly compute the maximum tolerable packet loss, latency, or required safety margin. Third, it serves as the mathematical backbone for the SMART implementation, guiding the calculation of adaptive braking thresholds that ensure safety under varying network conditions.

Corollary (100% Crash probability does not depend on p).

If the initial parameters allow for a 100% crash probability, then this threshold is always reached with the same τ independently of $p \in [0, 1)$.

$$\exists \tau, P_{crash}(\tau, p \in [0, 1)) = 1 \implies P(\tau, p) = 1 \forall p \in [0, 1], \tau \geq \frac{\Delta t}{2}$$

This result emphasizes that in latency-critical teleoperation or autonomous braking systems, **network delay is the dominant factor in safety**, whereas packet reliability only plays a secondary role.

In practical terms, system designers must ensure that the maximum one-way latency τ is well below $\Delta t/2$ to guarantee safe operation.

Corollary (Expected Minimum Distance).

The expected minimum distance reached before stopping is

$$\mathbb{E}[d] = S - L - d_{brake} - \frac{2\mathbb{E}[t] \cdot v_{max}}{1000}$$

This corollary is fundamental in the analysis of the experimental results because it provides a clear and quantitative measure of how the theoretical model aligns with reality. By predicting the expected minimum distance the robot reaches before stopping, it allows for a direct comparison between the modeled behavior and the actual performance observed during tests. Any divergence between the expected and measured distances highlights the influence of real-world factors. Consequently, this corollary serves as a crucial benchmark for evaluating both the accuracy of the model and the effectiveness of the implemented safety strategies, enabling a rigorous assessment of system reliability under varying network conditions.

Corollary (SMART Algorithm).

Let $P_{target} \ll 1$ and $p \in (0, 1)$ denote the desired crash probability of a mobile robot under the smart implementation, then its ideal threshold should be:

$$S = \frac{v_{max}}{1000} \left(T(N^* + 1) + 2\tau \right) + L + m + d_{brake}$$

Where N^* is the first N that satisfies $P_{crash}(N) \leq P_{target}$.

The SMART implementation provides a systematic method to adjust the braking threshold based on network conditions and the desired crash probability. This approach allows the robot to start braking earlier when network impairments are present, helping to reduce the risk of collisions compared to a fixed-threshold strategy. While it does not guarantee perfect safety, it offers a more controlled and predictable response under non-ideal communication scenarios.

Sources for later

O'Brien, W. (2025). Performance Benchmarking of Private 5G Networks. *ScienceDirect*.
<https://www.sciencedirect.com/science/article/pii/S1389128625006358>

Latency (ms)	Nokia Core		Raemis Core	
	Un-loaded	Loaded	Un-loaded	Loaded
Mean	24.44	26.73	24.37	31.58
Standard Dev.	3.97	3.86	4.58	6.90
Skew	0.28	0.48	0.47	0.77
Kurt.	2.24	2.56	2.56	3.82
Min.	16.13	18.53	16.62	17.79
25%	21.30	23.57	20.69	26.76
50% (Median)	23.95	26.36	23.86	31.03
75%	27.34	28.93	27.23	35.05
Max	33.91	38.15	37.67	56.85

Data

Latency and Reliability Analysis of a 5G-Enabled Internet of Musical Things system Luca Turchet, Senior Member, IEEE, and Paolo Casari, Senior Member, IEEE
https://iris.unitn.it/retrieve/handle/11572/382069/656979/Latency_and_Reliability_Analysis_of_a_5G-Enabled_Internet_of_Musical_Things_system.pdf

TABLE II
MEAN, STANDARD DEVIATION, MINIMUM AND MAXIMUM OF THE FOUR
INVESTIGATED METRICS FOR EACH CONDITION. THE UTILIZED NMP
SYSTEM INVOLVED A JITTER BUFFER WITH SIZE OF ≈ 10.66 MS.

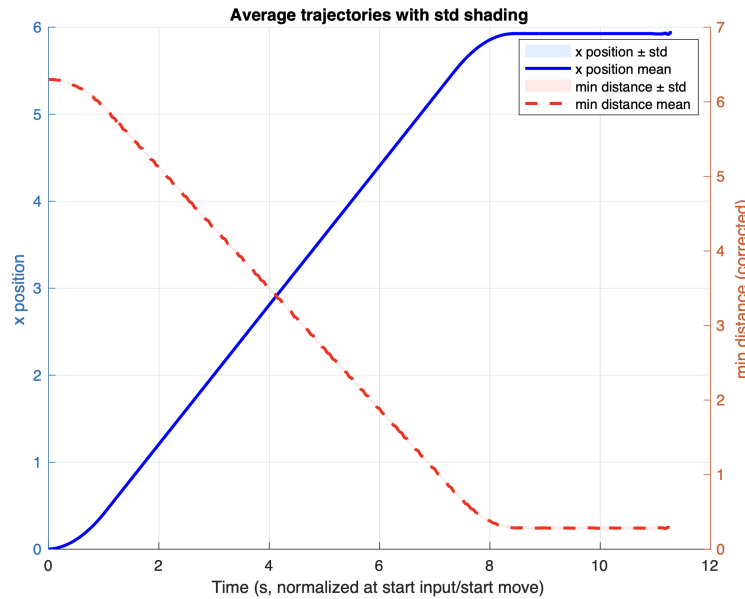
	Mean	SD	Min	Max
CONDITION 1: 2 boxes				
Latency (ms)	22.27	0.58	21.08	27.42
Packet loss ratio	0.0055	0.0065	0	0.0582
Missed packets	9.7	11.53	0	102
Max number of consecutive missed packets	2.63	8.14	0	93
CONDITION 2: "2 boxes + UDP"				
Latency (ms)	22.78	0.57	22.05	26.28
Packet loss ratio	0.0057	0.0092	0	0.056
Missed packets	10.03	16.1	0	98
Max number of consecutive missed packets	3.16	10.35	0	80
CONDITION 3: "2 boxes + TCP"				
Latency (ms)	23.68	0.61	23.02	28.93
Packet loss ratio	0.006	0.0081	0	0.046
Missed packets	10.63	14.32	0	82
Max number of consecutive missed packets	3.5	10.28	0	78
CONDITION 4: "3 boxes"				
Latency (ms)	22.36	0.51	20.85	25.84
Packet loss ratio	0.006	0.0076	0	0.0685
Missed packets	10.63	13.44	0	120
Max number of consecutive missed packets	2.95	7.71	0	93
CONDITION 5: "3 boxes + UDP"				
Latency (ms)	22.75	0.42	21.96	24.61
Packet loss ratio	0.0061	0.0075	0	0.0394
Missed packets	10.81	13.15	0	69
Max number of consecutive missed packets	2.33	3.75	0	43
CONDITION 6: "3 boxes + TCP"				
Latency (ms)	22.83	0.57	21.71	27.33
Packet loss ratio	0.0061	0.0083	0	0.0662
Missed packets	10.67	14.56	0	116
Max number of consecutive missed packets	2.83	7.77	0	99
CONDITION 7: "4 boxes"				
Latency (ms)	22.62	0.5	21.11	28.48
Packet loss ratio	0.0062	0.00711	0	0.0697
Missed packets	10.94	12.44	0	122
Max number of consecutive missed packets	3.3	7.96	0	102
CONDITION 8: "4 boxes + UDP"				
Latency (ms)	23.21	0.51	21.82	27.31
Packet loss ratio	0.0064	0.0074	0	0.0782
Missed packets	11.2	13.06	0	137
Max number of consecutive missed packets	2.87	5.93	0	94
CONDITION 9: "4 boxes + TCP"				
Latency (ms)	23.02	0.43	21.80	26.57
Packet loss ratio	0.0064	0.0075	0	0.0862
Missed packets	11.21	13.2	0	151
Max number of consecutive missed packets	3.18	7.38	0	99

Data 2

7) Simulation Results

First, the design parameters were chosen: **brake threshold** $S = 100$ [cm], **maximum speed** $v_{max} = 80$ [cm/s], and the **safety margin** $m = 5$ [cm] required at the end of braking.

The other parameters employed in building the model were obtained under **idealized conditions** ($\tau = p = 0$), where the communication link is assumed to be free of disturbances such as latency, jitter, or packet loss. Establishing the baseline performance in such an environment makes it possible to isolate the intrinsic dynamics of the robot and its control strategy. Subsequent simulations and experiments under degraded conditions can then be directly contrasted with this ideal case, allowing the impact of network impairments to be quantified with clarity.



Figure

The simulations conducted in this study produced quantitative metrics that characterize the dynamic behavior of the ROSbot XL under the tested conditions. Table 4.X summarizes the key numerical results obtained across all runs:

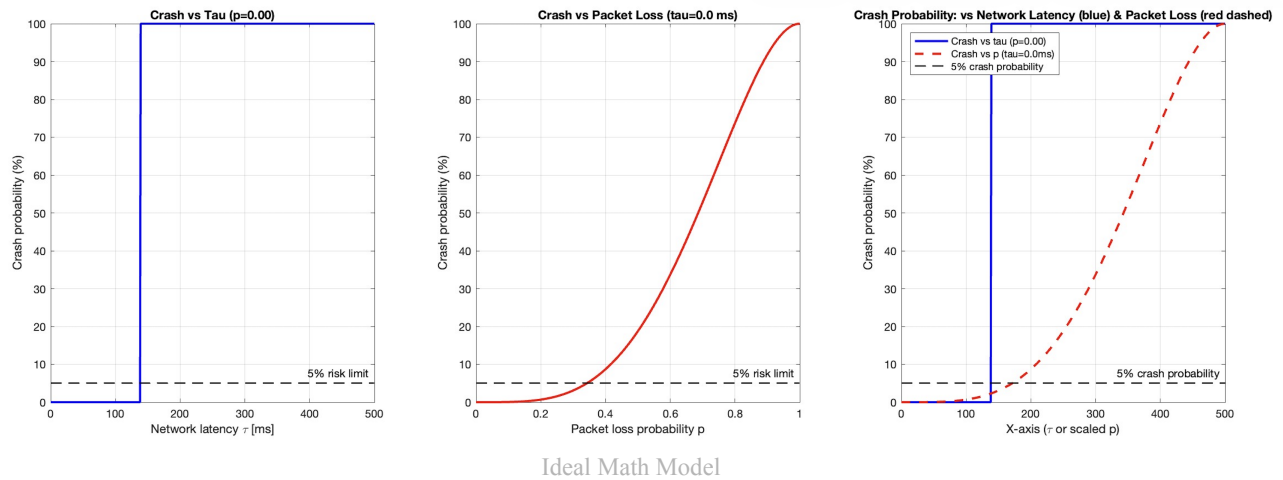
Metric	Value	Std	Unit
Top Speed	83.18	3.29	cm/s
Minimum Distance	28.38	2.13	cm
Stopping Distance	45.57	2.13	cm
Brake Time	1.14	0.02	s
Start Input Latency	0	0	ms
Expected Minimum Distance	25.45	109.18	cm

It is worth noting that the measured maximum speed of the robot differs from the design specifications. This deviation arises from inherent variations in the actuator performance of the robot's motors, which influence the actual achievable velocity in a real-world or simulated environment. The standard deviation of the expected minimum distance remains relatively high. This is because it is dominated by the natural variations in the robot's measured stopping distance, its top speed during each run, and the response time of the actuators. Even small fluctuations in these quantities propagate through the calculation and can amplify the overall variability.

In other words, although the delay and packet loss are fixed, the robot's motion exhibits inherent trial-to-trial difference slight differences in acceleration, braking behavior, or sensor measurement which accumulate and result in a higher observed standard deviation for the expected minimum distance. This variability reflects the realistic uncertainty present in the robot's physical performance rather than an error in the calculation itself.

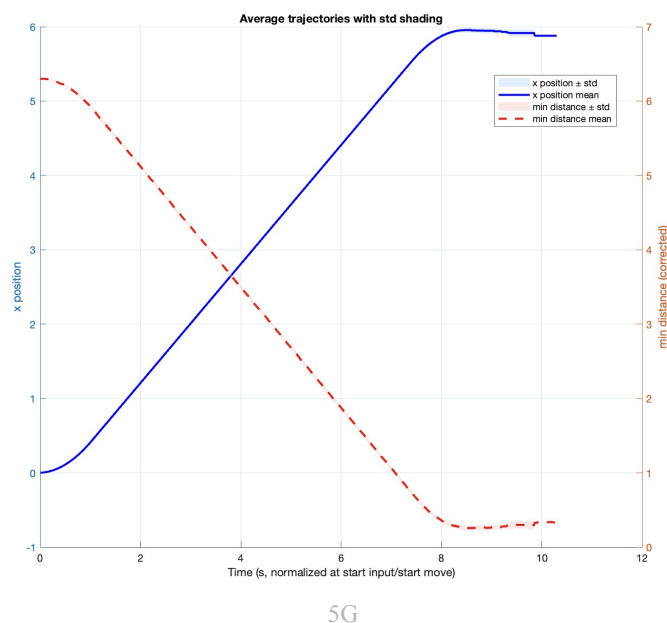
From the collected data, several graphical representations were generated to visualize the system behavior in accordance with the developed mathematical model. These graphs illustrate the relationship between speed, braking distance, and minimum distance to obstacles, and they provide insights into the dynamic response of the robot under controlled stopping scenarios.

Notice how the maximum speed differs from the one set in the design parameters as it varies based on the actuators in the robot motors.



Overall, the simulation results demonstrate that the implemented motion model closely aligns with theoretical predictions, validating the modeling approach and providing a baseline for further experiments involving network-induced disturbances and event-triggered actuation.

The first network scenario evaluated corresponds to a worst-case 5G communication environment. In this scenario, the network exhibits an average latency of 32 ms and an average packet loss of 0.6%. Despite being considered a “worst-case” configuration, these parameters still represent near-ideal conditions for robotic control applications. Consequently, the results obtained under this scenario closely match the predictions of the theoretical model, with minimal deviations observed in the robot’s dynamic response. This outcome confirms that, under high-performance 5G connectivity, the system’s behavior remains largely unaffected by network-induced disturbances, validating the robustness of the control and actuation strategies in low-latency, low-loss environments.



Metric	Value	Std	Unit
Top Speed	81.99	3.15	cm/s
Minimum Distance	25.83	3.61	cm

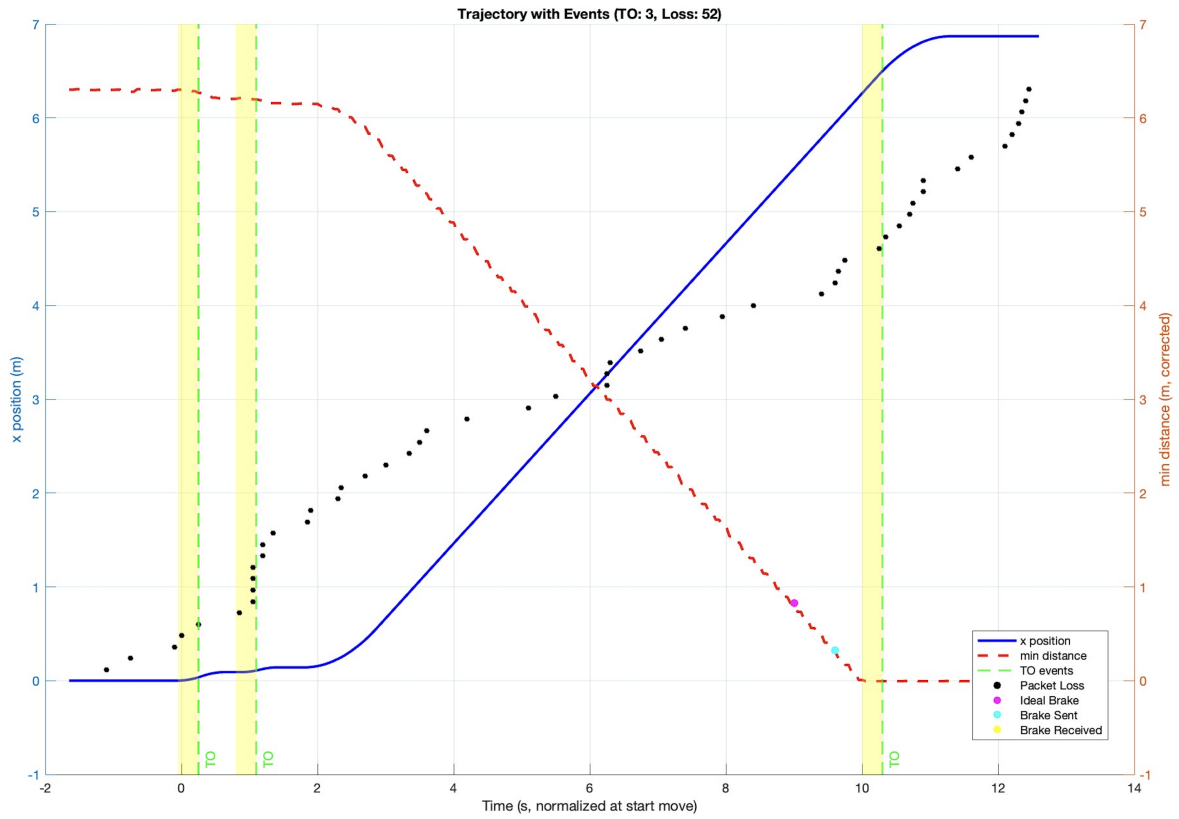
Metric	Value	Std	Unit
Stopping Distance	46.57	1.27	cm
Brake Time	1.158	0.03	s
Start Input Latency	17.78	26.82	ms
Expected Minimum Distance	25.60	104.46	cm

The measured Start input latency corresponds directly to the one-sided network delay, as expected. The relatively high standard deviation observed across simulation runs is attributed to variations in the initiation of the command send period. Specifically, the send period does not start uniformly in all simulations due to CPU scheduling limitations, introducing slight temporal inconsistencies that manifest as variability in the measured latencies. This behavior highlights the influence of computational factors on real-time performance, even under otherwise ideal network conditions.

It is important to note that the only instance in which the measured stopping distance and brake time directly correspond to the physical parameters of the robot occurs when braking is initiated under ideal conditions. In practice, however, these metrics are more meaningfully evaluated by considering the interval between the moment at which the robot *should ideally* begin braking and the moment at which it actually comes to a stop. This approach captures the combined effects of system delays, control latency, and communication imperfections, providing a more realistic and insightful assessment of the robot's performance. Consequently, analyzing the stopping distance and brake time in this manner offers a richer characterization of the system's responsiveness and operational reliability.

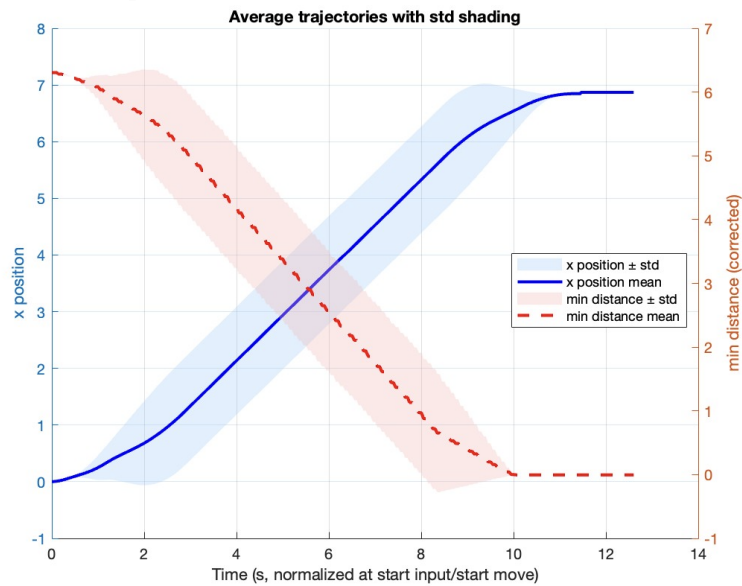
The second network scenario considers a worst-case satellite communication link, where the only available channel is provided by a geostationary satellite at an altitude of 35,000 km. This configuration introduces a substantial one-sided latency of approximately 600 ms, which greatly exceeds the maximum allowable delay, $\Delta t/2$. Additionally, the network exhibits a high packet loss probability of 15%. However, according to the theorem on 100% crash probability, such packet loss does not alter the catastrophic outcome observed in these runs. In other words, the overwhelming impact of the excessive delay dominates the system behavior, rendering the packet loss secondary in determining the robot's failure under these extreme conditions.

Considering a single simulation run, the extreme number of packet losses and the pronounced effects of the prolonged timeouts are clearly observable. This example vividly illustrates how severe network impairments can directly impact the robot's response, highlighting the interplay between high latency, packet loss, and system performance under adverse conditions.



Single Run

Moreover, the effect of the delay is clearly noticeable: the ideal braking point differs drastically from the actual moment at which the brake command was transmitted. In addition, the absence of a recorded point for the instant when the brake signal was received is particularly significant. This missing event indicates that, in practice, the signal would only have been processed after the abrupt crash had already occurred, further emphasizing the severity of the network-induced impairment.



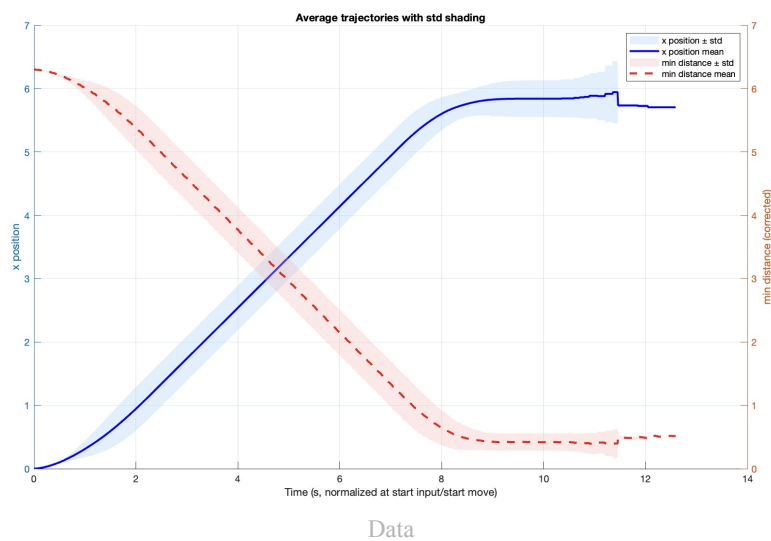
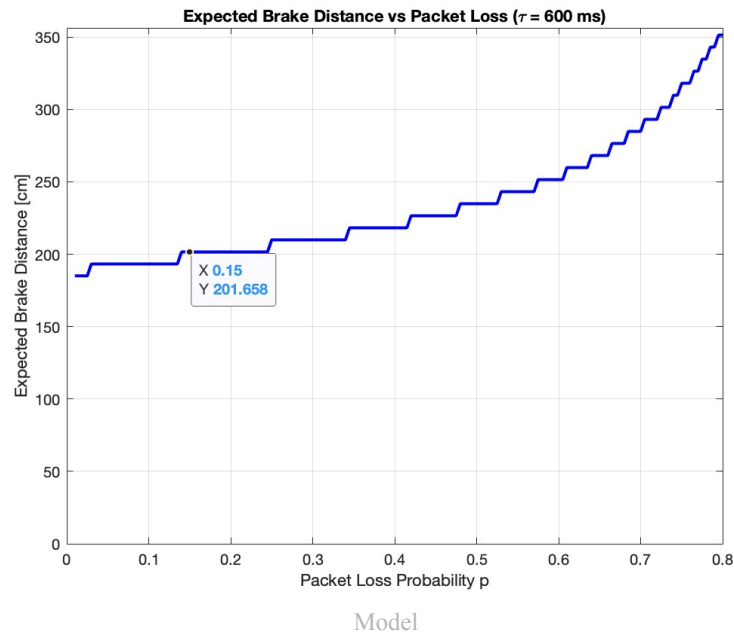
Full Satellite

Metric	Value	Std	Unit
Top Speed	83.45	3.9	cm/s
Minimum Distance	0	0	cm
Brake Time	1.66	0.08	s
Start Input Latency	1432.67	57.45	ms

Metric	Value	Std	Unit
Expected Minimum Distance	-71.93	x	cm

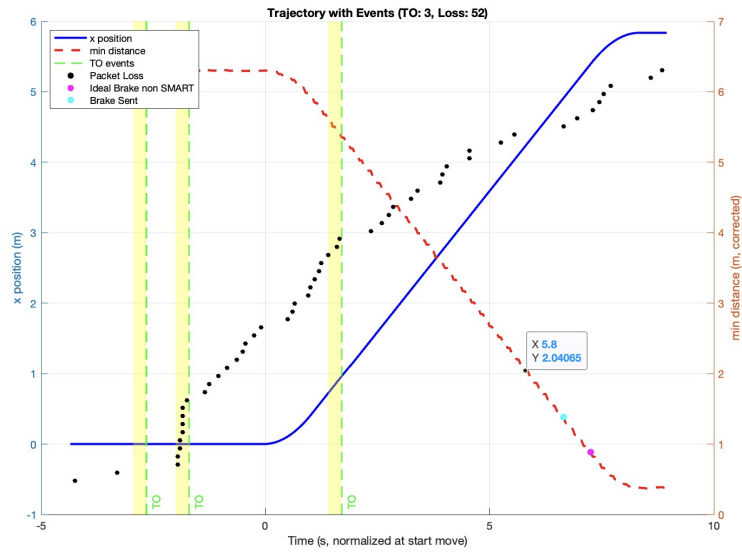
In this scenario, the measured brake time is not meaningful, as the robot fails to initiate braking and consequently collides. Conversely, the expected minimum distance predicted by the model results in a value of -72 cm, unequivocally indicating a collision. This highlights the predictive capability of the model in identifying unsafe conditions, even in cases where direct measurements cannot provide relevant insight.

This case therefore provides an ideal candidate for evaluating the Safe Margin Automatic Response Tool (SMART) implementation, as it represents the exact class of catastrophic conditions that SMART is designed to mitigate.



Metric	Value	Std	Unit
Top Speed	83.69	3.38	cm/s
Minimum Distance	44.71	6.32	cm
Brake Time	1.69	0.059	s
Start Input Latency	1346.37	458.4	ms
Expected Minimum Distance	29.82	x	cm

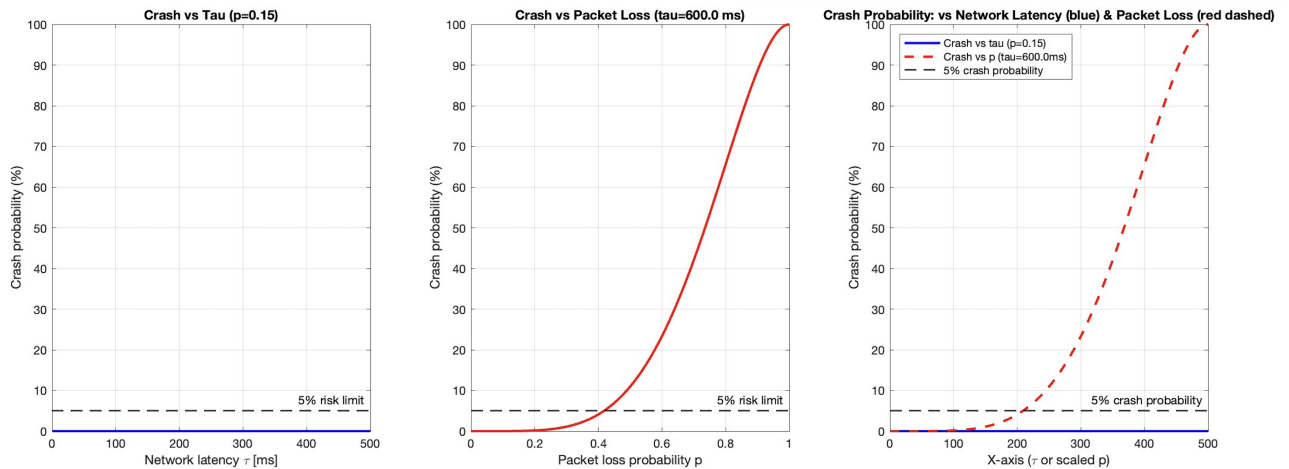
This specific case is particularly instructive, as it highlights how the SMART implementation successfully triggers a braking action before the non-SMART scenario. Moreover, it illustrates how a combination of packet losses and intermittent timeouts can substantially delay the initiation of the robot's motion, emphasizing the critical impact of network reliability on system performance.



Special Case

The SMART model provides a more conservative estimate of the minimum distance compared to measurements obtained from the actual robot. The discrepancy observed between the mathematically computed safe distance S and the empirical measurements can be mainly attributed, to the discrete nature of the iteration parameter N . The model calculates the brake threshold based on a target safety probability P_{target} , but because N can only take integer values, the computed probability $P(N)$ will always satisfy $P(N) \leq P_{target}$ as stated in the theorem. This discretization introduces a small but systematic overestimation of the required stopping distance, making the model inherently safer.

In fact, by analyzing the mathematical model with updated brake thresholds and network parameters, it becomes clear how this bound is effectively lowered.

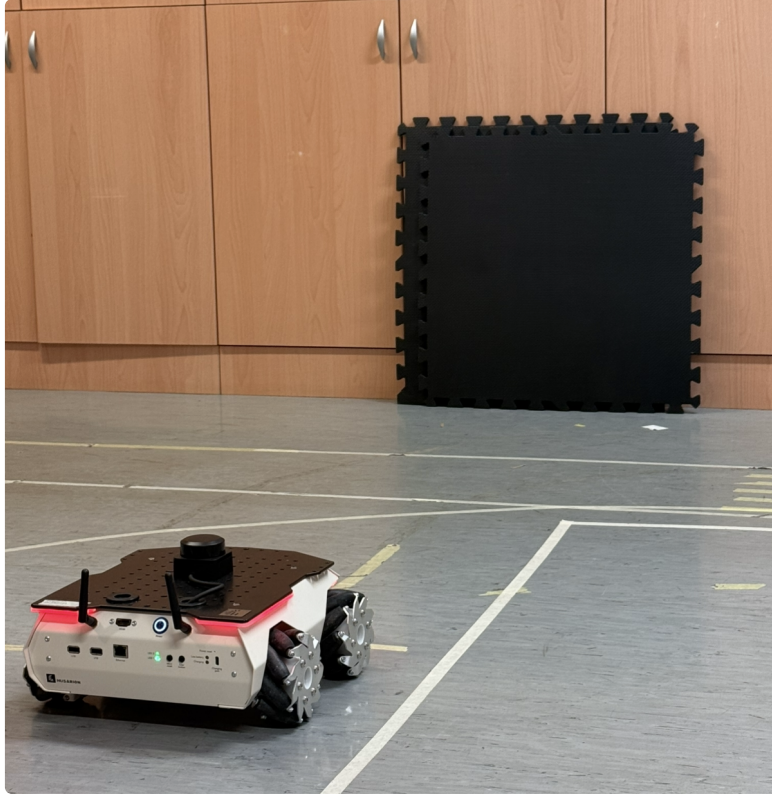


Updated Model

TODO 3G/LTE TESTS AND LESS AGGRESSIVE SATELLITES LIKE STARLINK

8) Real Robot Experiments

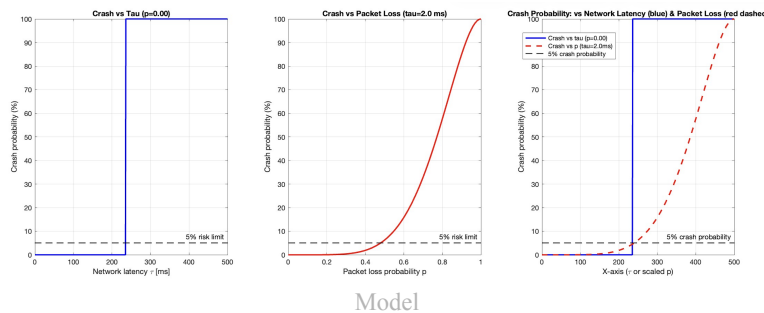
Experiments conducted on the physical ROSbot XL differed slightly from those performed in simulation. For safety reasons, the robot's design parameters were adjusted to $L = 12, [cm]$ and $v_{\max} = 50, [cm/s]$ to minimize potential damage in the event of a collision. Unlike the simulation, no failsafe timeout (TO) was implemented, and network impairments such as packet loss and latency were emulated in software rather than through the *netns* network stack. Additionally, only the distances corresponding to the onset of braking and complete stop could be directly measured. Despite these differences, the experimental results closely approximated the ideal scenario, confirming the validity of the underlying model.



Testbench

As with the simulation, intrinsic robot parameters were determined through idealized test runs. Since a cabled connection was impractical, a private 5G network was employed for these baseline measurements. The simulated results indicate that, even under moderate network congestion, the measured performance deviates only slightly from the ideal case. Specifically, the measured braking distance was $d_{\text{brake}} = 9, [cm]$ in $t = 0.36 [s]$.

Based on these observations, the following model was formulated to describe the robot's braking behavior under real-world conditions:



The obtained minimum distance is of resembles the same expected minimum distance of $23.3 [cm]$.

To evaluate the system under extreme network conditions, a satellite connection with a one-way latency of $\tau = 320, [ms]$ and packet loss of $p = 15\%$ was simulated. This scenario represents a worst-case communication

environment, in which the latency significantly exceeds the robot's reaction time ($\tau \gg \Delta t/2$), making a collision unavoidable. Indeed, all experimental trials under these conditions resulted in a crash.

The SMART algorithm was applied, yielding an updated brake threshold of $S = 87$, [cm]. Under this configuration, the expected stopping distance was $\mathbb{E}[d] = 40.1$, [cm], while the experimentally observed distances averaged $d = 46$, [cm]. The discrepancy between the expected and measured values can be attributed both to minor variations in test conditions and to the inherent limitations of the robot's sensors and actuators, which introduce additional variability in real-world performance.

Finally, experiments were conducted simulating Starlink connectivity, with a one-way latency of $\tau = 50$ [ms] and a packet loss of $p = 2\%$. This latency remains below the critical crash threshold, although the connection exhibits some instability. Under these conditions, the mathematical model predicts a final stopping distance of 18.3 [cm], which is in close agreement with the empirical measurements, averaging 18 [cm]. In a worst-case Starlink scenario, where packet loss can increase to approximately 10% , the experimental results still align well with the model, confirming its predictive reliability.

To further explore the stability of the system near its operational limits, an additional test was performed with a latency close to the theoretical threshold, $\tau = 200$ [ms] $\lesssim \Delta t/2$. In this configuration, the model predicts a stopping distance of 8 [cm], which was consistently observed in experiments. This outcome is consistent with the expectation that, in the absence of packet loss, the system maintains a near-zero crash probability, validating both the robustness and accuracy of the modeling approach.

9) Final Comparison and Evaluation

Todo

10) Appendix A: Setup and Numerical Analysis Implementation

To evaluate the impact of network impairments on teleoperation, a controlled network environment is created using *Linux namespaces (netns)*. Namespaces provide logically isolated network stacks, enabling the emulation of multiple, independent networks on a single physical machine. This approach allows precise control over delays, packet loss, and other impairments, without requiring external hardware or impacting the host system's main network.

In this work, three primary namespaces are created:

1. **Robot Namespace (`ns_robot`)** – contains the ROS 2 nodes and the Gazebo simulation representing the ROSbot XL. This namespace simulates the network environment as seen by the robot.
2. **Controller Namespace (`ns_ctrl`)** – contains ROS 2 nodes representing the human operator or control station. Start/stop commands and other teleoperation instructions are generated from this namespace.
3. **Data/Logging Namespace (`ns_data`)** – optionally used for data collection, logging, or monitoring network statistics in isolation from the robot and controller namespaces.

Virtual Ethernet (veth) pairs connect these namespaces, effectively creating point-to-point links. Each link can be independently configured with:

- **Delay (τ)** – to emulate transmission latency.
- **Packet loss (p)** – to reproduce unreliable links.

Once the namespaces are configured, ROS 2 nodes are launched in their respective namespaces. The robot simulation in Gazebo runs within `ns_robot`, while the operator nodes execute in `ns_ctrl`. This separation ensures that network impairments affect only the communication between the robot and the controller, providing an isolated and reproducible testing environment. Data collected from `ns_data` allows precise measurement of latency, packet loss, and other network metrics, which are later used for model validation and performance analysis.

The ROSbot XL is equipped with a 2D LiDAR sensor that provides distance measurements across a wide angular range. To reduce computational load and focus on the most relevant obstacle information, the LiDAR data is first **decimated** by selecting a subset of angles corresponding to the robot's forward path.

From this reduced set of measurements, the **minimum distance** is extracted and used as a conservative safety estimate. By considering the closest detected object, the system ensures that the computed safe distance always reflects the worst-case scenario, thereby maximizing operational safety during start/stop teleoperation.

This processing pipeline allows the control algorithms to react promptly to potential collisions while maintaining a manageable data rate for real-time computation.

```
self.k = 100 # angle based on LiDAR sensibility
first_k = lidar.ranges[:self.k]
last_k = lidar.ranges[-self.k:]
combined = first_k + last_k
valid = [
    r for r in combined
    if math.isfinite(r) and msg.range_min < r < msg.range_max
]
distance=min(valid)
```

The SMART implementation employs an iterative algorithm to determine the safe stopping distance based on real-time network conditions and predicted packet loss. The procedure continuously evaluates whether the robot can safely continue its motion or if a stop command must be issued to avoid collisions.

```

# Parameters
m = 5                # safety margin (cm)
bot_length = 18      # bot length (cm)
d_brake = 45.57      # braking distance (cm)
vmax = 83.18         # max speed (cm/s)
T = 100             # control period (ms)
P_crash_target = 0.05 # target crash probability
# Inputs
packet_loss = self.packet_loss / 100.0 # in 0-1
avg_latency = self.avg_latency         # in ms

# Safety check to avoid log(0) or division by zero
p = min(max(packet_loss, 1e-6), 1-1e-6)

# Step 1: Compute minimal N* satisfying crash probability
N = 0
while N<20:
    P_crash = p*N * (1 + (1-p)*N)
    if P_crash <= P_crash_target:
        N_star = N
        break
    N += 1

# Step 2: Compute safe distance
S = d_brake + m + bot_length + (vmax / 1000) * (T*(N_star + 1) + 2*avg_latency)

self.safe_distance = S / 100

```

For the parameters used in this work, the number of iterations was limited to `N<20`. This choice ensures that the algorithm satisfies a target safety probability of $P_{target} = 0.05$ even under the worst-case packet loss scenario of $p = 0.5$. By bounding `N`, the algorithm maintains computational efficiency while providing a probabilistic guarantee of safety, making it suitable for real-time deployment on the ROSbot XL.

11) Appendix B: Proofs

This appendix presents rigorous proofs of the theorems and corollaries employed in the mathematical model. The proofs are provided in the order in which the corresponding theorems and corollaries appear in the main text. Including these proofs serves two purposes: (i) to demonstrate the correctness and internal consistency of the mathematical framework underlying the study; and (ii) to provide a complete reference for readers who wish to understand or reproduce the derivations that support the model's predictions.

Proof of Expected Crash Bound:

The bot doesn't have to stop at the distance threshold to avoid collisions, but it has to stop before the critical distance $d_{max} = m + d_{brake}$. From here the maximum distance the bot can still safely travel after the threshold is $\Delta d = S - d_{max}$. The time it takes to travel said distance is

$$\Delta t = 1000 \cdot \frac{\Delta d}{v_{max}}$$

The two messages can be modeled as geometric random variables where the success probability to successfully deliver the i -th packet at time $t_i = \tau + iT$ is given by

$$P(t_i = \tau + iT) = p^i(1 - p)$$

Therefore the expected time for the packet to arrive is:

$$\mathbb{E}[t_{received}] = \sum_{n=0}^{\infty} P(t_n = \tau + nT) = \sum_{n=0}^{\infty} p^n(1 - p) = \sum_{n=0}^{\infty} (nT + \tau)p^n(1 - p) = \tau + \frac{pT}{1 - p}$$

However this doesn't account the discretization of the signals given by the send period T as it will differ from the time the bot should send the data and when it will be sending it. This has expected value of $\mathbb{E}[t_{processing}] = T/2$

Finally we have

$$\mathbb{E}[t] = \mathbb{E}[t_{received}] + \mathbb{E}[t_{processing}] = \tau + T \cdot \frac{1 + p}{2(1 - p)}$$

Since the brake signal is sent only after the delivery of the laser sensor data this time must be doubled to obtain the correct bound.

□

Proof of Crash probability Theorem:

As in the previous case, the maximum allowed time delay is Δt . From this, we can determine the maximum number of packets that can be safely sent:

$$N(\tau) = \left\lfloor \frac{\Delta t - 2\tau - T}{T} \right\rfloor + 1 = \left\lfloor \frac{\Delta t - 2\tau}{T} \right\rfloor$$

the $+1$ term is used as $\exists P_{crash} \neq 1 \iff N \neq 0$.

Let $X, Y \sim \text{Geom}(1 - p)$ be two geometric random variables representing the number of attempts required for successful delivery of the sensor packet and the brake packet, respectively, and let n, m denote their corresponding number of tries. A crash occurs if

$$Z = X + Y > N$$

Which is equivalently formulated as

$$n + m \leq N$$

It is important to note that X and Y are not independent, as Y cannot succeed until X has achieved at least one successful delivery. The probability of a crash can then be expressed as

$$P(X + Y > N) = \sum_{n=0}^{N-1} [P(X + Y > N | X = n)P(X = n) + P(X > N)]$$

Consequently, a crash can occur in two distinct scenarios:

$$P_{\text{crash}} = P_{\text{data fail}} + P_{\text{brake fail}}$$

- **Data Fail:** This corresponds to the event where the sensor data packets never reach the controller within the allowed time. Formally, this probability is given by:

$$P_{\text{data fail}} = \sum_{n=0}^{N-1} P(X > N) = \sum_{n=0}^{N-1} p = p^N$$

- **Brake Fail:** This occurs when the sensor data packet successfully reaches the controller, but the brake command from the controller does not arrive within the desired time frame. Let $n \leq N$ denote the first successful sensor packet. Then the stop signal can be sent at most $m = N - n$ times. The associated probabilities are

$$P(X = n) = p^n(1 - p)$$

$$P(X + Y > N | X = n) = P(Y > N - n) = p^{N-n}$$

From here

$$P_{\text{brake fail}} = \sum_{n=0}^{N-1} P(X + Y > N | X = n)P(X = n) = \sum_{n=0}^{N-1} p^n(1 - p)p^{N-n}$$

$$= \sum_{n=0}^{N-1} (1 - p)p^N = (1 - p)p^N N$$

And finally

$$P_{\text{crash}}(p, \tau) = p^{N_\tau} (1 + (1 - p)N_\tau)$$

□

Proof of 100% Crash Probability Does Not Depend on p Corollary:

Clearly for $p = 1$ we always crash

Clearly a 100% crash probability happens also if $N(\tau) = 0$, then, by selecting the lower bound the result is obtained

$$N(\tau) = 0 = \frac{\Delta t - 2\tau_p}{T} \rightarrow \tau_{100\%} = \frac{\Delta t}{2}$$

where the $+1$ term was removed as this N must allow for a zero.

□

Proof of Expected Minimum Distance Corollary:

As before let $X, Y \sim \text{Geom}(1 - p)$ be two geometric random variables representing the number of attempts required for successful delivery of the sensor packet and the brake packet, respectively. Then the time to fully brake is:

$$t_{\text{brake}} = XT + \tau + YT + \tau$$

And the expected time is

$$\mathbb{E}[t_{\text{brake}}] = \mathbb{E}[(X + Y)T + 2\tau] = T(\mathbb{E}[X] + \mathbb{E}[Y]) + 2\tau = 2T \frac{p}{1 - p} + 2\tau = \mathbb{E}[t_{\text{success}}]$$

As before, add the processing delays due to T and $\mathbb{E}[t_{\text{brake}}] = \mathbb{E}[t]$

□

Proof of SMART Algorithm:

Notice that P_{crash} is strictly decreasing:

$$P_{crash}(N) - P_{crash}(N+1) = p^N(1-p^2)(N+1) > 0$$

And has the following limits

$$P_{crash}(0) = 1 > P_{target} \quad \lim_{N \rightarrow \infty} P_{crash}(N) = 0 < P_{target}$$

Therefore $\exists N^* \in \mathbb{R}^+ : P_{crash}(N^*) = P_{target}$ and the optimal value is the first obtained due to monotonicity.

□